

SUTDoko: Denoising Language Models for Room Address Lookup

G. Lim, E. Ang, M. Shaswat, M. Ramaswamy

April 2026

- SUTD has over 200 rooms across 5 buildings.
- Rooms are often communicated by name.
- Room-to-address lookup can be a hassle.

- Resources for this task remain limited.
- Physical signs often require one to walk around.
- Digital resources like the SUTD Assistive Maps (SAM) Telegram bot (Telegram) often fail on noisy input (e.g. strings with typos).

- We introduce SUTDoko to address this gap.
- Visit **sutdoko.vercel.app** for our web application.

Definition (Keyboard)

$$K = \begin{bmatrix} \text{~} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 0 & - & = \\ q & w & e & r & t & y & u & i & o & p & [&] & \backslash \\ & a & s & d & f & g & h & j & k & l & ; & ' & \\ & & z & x & c & v & b & n & m & , & . & / & \\ & & & & & & \square & \square & \square & \square & \square & & \end{bmatrix}$$

- We define a 60% ANSI QWERTY en-US keyboard layout (Wikipedia contributors, 2026).

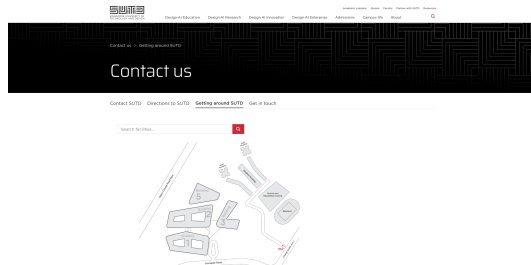


Figure: Official SUTD contact pages.

- We obtain a room set R by politely scraping official SUTD contact pages.
- We queried the facility search function by enumerating buildings and floors.

Let V be a vocabulary, and $R \subseteq V^*$ be a room set, and $\prec$ be a total order on R , and $N \in \mathbb{N}_{\geq 1}$ be a result count, and $\prec_N$ be a total order on $R_{\prec}^N$, and $A \subseteq V^*$ be an address set, and $B \subseteq V$ be a boundary set, and $C \in \mathbb{N}$ be a corruption count, and $\alpha \in \mathbb{R}_{[0,1]}$ be a transposition rate, and $\beta \in \mathbb{R}_{[0,1]}$ be a substitution rate, and $\boldsymbol{\pi} = (\pi_{\text{train}}, \pi_{\text{val}}, \pi_{\text{test}}) \in \Delta_C^2$ be a training-validation-testing split, and $I : V \rightarrow \mathbb{Z}^2$ be a keyboard-position map.

Algorithm 1 CORRUPT($\mathbf{x}$)

Require: $\mathbf{x} \in V^*$ $(T_L, T_M, T_R) \leftarrow (V^*B \cup \{()\}, (V \setminus B)^+, BV^* \cup \{()\})$ $T \leftarrow \{uwbvz \mid u \in T_L, v, w \in T_M, z \in T_R, b \in B, \mathbf{x} = uvbwz\} \setminus \{\mathbf{x}\}$ **if** $T \neq \emptyset$ **then** $\mathbf{x} \sim \alpha \mathcal{U}(T) + (1 - \alpha) \delta(\mathbf{x})$ **for** $i \in [1..|\mathbf{x}|]$ **do** $S \leftarrow \{k \in K \setminus \{x_i\} \mid I(k) - I(x_i) \in [-1..1] \times [-2..2]\}$ **if** $S \neq \emptyset$ **then** $x_i \sim \beta \mathcal{U}(S) + (1 - \beta) \delta(x_i)$ **return** $\mathbf{x}$

- We generated corrupted strings and split corrupted-uncorrupted room pairs into training, validation, and testing datasets:
 - Our corruption algorithm transposes spans and substituting characters.
 - We set $C = 50$, and $\alpha = 0.5$, and $\beta = 0.05$, and $\pi = (0.8, 0.1, 0.1)$ for tractable and plausible-looking typos.
 - We stratify the split by room to make the datasets comparable across rooms.

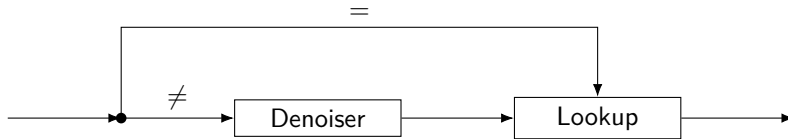


Figure: Inference algorithm.

Definition (Denoiser)

Let $f : V^* \times R \rightarrow \mathbb{R}$ be a scorer.

$$\mathcal{D}_f : V^* \rightarrow R$$
$$\mathcal{D}_f(x) = \min_{\prec} \arg \max_{r \in R} f(x, r)$$

- Our denoiser is a Nanochat-parameterised (Karpathy, 2025), character-level transformer of depth 2 with decoding constrained to the character-level trie induced by R .
 - Character-level tokenisation: As room strings/typos are often short/character-level.
 - Trie-constrained decoding: To efficiently avoid room hallucination.
- We train our denoiser using a standard cross-entropy loss and AdamW optimiser.¹

¹A more complete list of hyperparameters can be found in our code repository.

Definition (Lookup)

$$\mathcal{L} : R \rightarrow A$$

- Our lookup is a standard room-address table.

Denoiser	Accuracy (%)	Mean Latency (ms)
LCP	29.2	0.04
DL	71.9	16.7
SJ	89.3	20.8
Ours	97.3	8.19

Table: Denoiser evaluation results summary.

- We evaluated our denoiser against substring Jaccard similarity (SJ), negative Damerau–Levenshtein distance (DL), and longest common prefix length (LCP) denoisers.
- We measured accuracy and mean latency of each denoiser on test set.
- We fixed a seed and hardware for fairness and reproducibility.

Algorithm 2 PREDICT(x)

Require: $x \in V^*$ $x \leftarrow \text{ALIAS}(\text{ALPHANUMERIC_SPLIT}(\text{LOWERCASE}(\text{DEPUNCTUATE}(\text{STRIP}(x))))))$ **if** $x \in R$ **then return** $((x, \mathcal{L}(x)))$ $f \leftarrow (\text{LCP}, \text{SJ}, \text{DL}, \text{OURS})$ **for** $r \in R$ **do** **for** $f \in f$ **do** $p(r, f) \leftarrow |\{r' \in R \mid f(x, r') > f(x, r)\}| + 1$ **for** $r \in R$ **do** $s(r) \leftarrow \sum_{i=1}^{|f|} (1/|R|^{i-1})(|R| - p(r, f_i))$ $o \leftarrow \min_{\prec_N} \arg \max_{r \in R_N} \sum_{i=1}^N s(r_i)$ **return** $((o, \mathcal{L}(o)) \mid o \in o)$

- Our system makes a prediction by:
 - ① Normalising by stripping whitespace and punctuation, lowercasing, separating letters from numbers, and replacing words with their aliases.
 - Example: $\sqcup \text{LT}, 2 \sqcup \rightarrow \text{LT}, 2 \rightarrow \text{LT2} \rightarrow \text{1t2} \rightarrow \text{1t } 2 \rightarrow \text{lecture theatre } 2$.
 - ② Denoising and looking up with harmonically weighted Borda ranking.

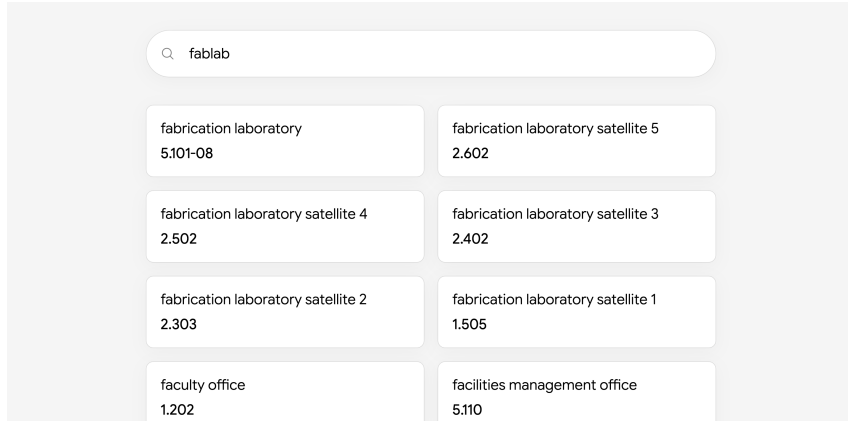


Figure: Web application.

- To further improve user experience by making our web application more efficient and responsive, we implemented:
 - **Incremental prediction.** We predict during typing rather than waiting for final submission.
 - **Prefix caching.** We KV-cache previously computed prefix states and only process newly appended characters.
 - **Debounced querying.** We introduce a short debounce to avoid redundant intermediate inference on rapid keystrokes.
 - **Redundant-query suppression.** We skip empty, already displayed, and exact-match queries when full recomputation is unnecessary.
 - **Bounded result caching.** We memoise recent solved queries while capping cache growth to prevent performance degradation over time.

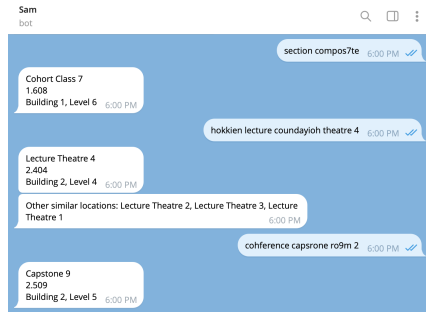


Figure: SAM.

- We evaluated our system against SAM.
- We measured accuracy on a held-out set of 10 inputs sampled from our test set.
- We fixed the same seed and hardware as before for fairness and reproducibility.

Input	design bdhavioural laboratory 1
Gold	behavioural design laboratory 1
Prediction	–
Input	capstone rolm 8
Gold	capstone room 8
Prediction	capstone room 8, capstone room 7, capstone room 6, capstone room 5
...	...
Input	cohference capsrone ro9m 2
Gold	capstone conference room 2
Prediction	capstone room 9

Table: SAM evaluation results.

System	Accuracy (%)
SAM	40
Ours	100

Table: System evaluation results summary.

- SAM occasionally recovered the target, but degraded as noise became more severe.
- Our system outperformed SAM, achieving complete recovery.
- We believe these results demonstrate strong promise and provide practical foundations for reliable campus assistance.

Notation

Let $n \in \mathbb{N}_{\geq 1}$.

$$R_{\prec}^n = \{\mathbf{r} \in R^n \mid r_1 \prec \cdots \prec r_n\}$$

Notation

Let $c \in \mathbb{N}$.

$$\Delta_c^2 = \left\{ \mathbf{p} \in \mathbb{R}_{[0,1]}^3 \mid \sum_{i=1}^3 p_i = 1, c\mathbf{p} \in \mathbb{N}^3 \right\}$$

Andrej Karpathy. nanochat: The best chatgpt that \$100 can buy.

<https://github.com/karpathy/nanochat>, 2025.

Telegram. Telegram: Launch @sutdmapbot. URL <https://t.me/SUTDMapBot>.

Wikipedia contributors. Qwerty. <https://en.wikipedia.org/wiki/QWERTY>, 2026.